Week 13: Localization — Bayes, Kalman, Particle Filter & MCL

EE0849 Introduction to Robotics — Detailed Notes

Basri Erdogan

Table of contents

1 How to Read These Notes

This document develops the **mathematics of four closely related ideas**, each of which we touched on in the W13 slides:

1. **Bayes filter** — the master equation.
2. **Kalman filter** — Bayes filter when everything is Gaussian and linear.
3. **Particle filter** — Bayes filter when the belief is represented by random samples.
4. **Monte Carlo Localization (MCL)** — particle filter applied to robot localization on a known map.

Every chapter alternates *math* with shaded *Intuition* boxes that re-state the equations in plain language. Read them in order — the four methods build on each other.

1.1 Notation Conventions

Symbol	Meaning
x_t	Robot **state** at time t, e.g. pose (x, y, θ)
u_t	**Control** applied between $t-1$ and t, e.g. (v, ω)
z_t	**Measurement** observed at time t, e.g. a LiDAR scan
$z_{1:t}$	Shorthand for the whole history $z_1, z_2, \dots, z_t$
$\text{bel}(x_t)$	**Belief** — probability distribution over x_t given everything so far
$\overline{\text{bel}}(x_t)$	Predicted belief, **before** incorporating z_t (bar = before measurement)
$p(A \mid B)$	Probability of event A **given** that B is known
$\mathcal{N}(\mu, \Sigma)$	Gaussian distribution with mean μ and covariance Σ
η	Normalization constant (chosen so a distribution integrates to 1)
$x_t^{[i]}$	The i-th **particle** (one sample) of pose x_t, $i = 1, \dots, N$
$w^{[i]}$	The **weight** of particle i

2 The Bayes Filter

2.1 The Problem Statement

A robot wants to estimate its current state x_t from:

- Its sequence of **controls** $u_{1:t} = (u_1, u_2, \dots, u_t)$
- Its sequence of **measurements** $z_{1:t} = (z_1, z_2, \dots, z_t)$

We never know x_t for sure — sensors are noisy and motion is uncertain. The most honest thing the robot can produce is a **probability distribution** over possible states, called the **belief**:

$$\boxed{\text{bel}(x_t) \;=\; p(x_t \mid z_{1:t},\, u_{1:t})}$$

Intuition

Read aloud: "the probability that the robot is at x_t, **given** everything I've seen and done so far." A sharp belief means high confidence; a flat one means almost any state is possible.

2.2 Two Assumptions That Make the Math Tractable

The Bayes filter relies on two **independence assumptions** that hold for most robots:

(A1) Markov assumption on the state. The next state depends only on the previous state and the latest control:

$$p(x_t \mid x_{0:t-1}, u_{1:t}, z_{1:t-1}) \;=\; p(x_t \mid x_{t-1}, u_t).$$

This conditional distribution is called the **motion model**.

(A2) Markov assumption on measurements. The current measurement depends only on the current state:

$$p(z_t \mid x_{0:t}, u_{1:t}, z_{1:t-1}) \;=\; p(z_t \mid x_t).$$

This is called the **measurement model** (or **sensor model**).

> Intuition
>
> (A1) says "the past is summarized by the present state" — if I know exactly where the robot is now and what control I'm applying, I don't need its history to predict the next position. (A2) says "what the sensor sees depends only on where I am right now," not on how I got here.

2.3 Deriving the Two Update Steps

We want a **recursive** formula — one that turns $\mathrm{bel}(x_{t-1})$ into $\mathrm{bel}(x_t)$ using only u_t and z_t.

2.3.1 Step 1: Predict (Motion Update)

Define the **predicted belief**:

$$\overline{\mathrm{bel}}(x_t) \;\equiv\; p(x_t \mid z_{1:t-1}, u_{1:t}).$$

Apply the law of total probability over the unknown previous state x_{t-1}:

$$\overline{\mathrm{bel}}(x_t) \;=\; \int p(x_t \mid x_{t-1}, z_{1:t-1}, u_{1:t})\; p(x_{t-1} \mid z_{1:t-1}, u_{1:t})\; \mathrm{d}x_{t-1}.$$

Using (A1) on the first factor (the past z's and earlier u's drop out) and the fact that u_t has not yet influenced x_{t-1}:

$$\boxed{\overline{\mathrm{bel}}(x_t) \;=\; \int p(x_t \mid u_t, x_{t-1})\; \mathrm{bel}(x_{t-1})\; \mathrm{d}x_{t-1}}$$

> Intuition
>
> "For every possible previous state, ask the motion model where it could have gone, then add up those possibilities weighted by how likely each previous state was." The integral is a continuous version of "for each x_{t-1}."

2.3.2 Step 2: Correct (Measurement Update)

Now fold in z_t using **Bayes' rule**:

$$\mathrm{bel}(x_t) \;=\; p(x_t \mid z_{1:t}, u_{1:t}) \;=\; \frac{p(z_t \mid x_t, z_{1:t-1}, u_{1:t})\; p(x_t \mid z_{1:t-1}, u_{1:t})}{p(z_t \mid z_{1:t-1}, u_{1:t})}.$$

By (A2) the first factor becomes $p(z_t \mid x_t)$; the second factor is exactly $\overline{\text{bel}}(x_t)$; and the denominator does not depend on x_t — call its inverse η:

$$\boxed{\text{bel}(x_t) \;=\; \eta\, p(z_t \mid x_t)\, \overline{\text{bel}}(x_t)}$$

with η chosen so $\int \text{bel}(x_t)\, \mathrm{d}x_t = 1$.

Intuition

"Take the predicted belief and reshape it by the likelihood of what I just measured." States where the prediction agrees with the measurement get boosted; states that contradict the measurement get suppressed.

2.4 The Bayes Filter Algorithm

Putting both steps together:

```
function BayesFilter(bel_prev, u_t, z_t):
    for each candidate state x_t:
        bel_bar[x_t] =   p(x_t | u_t, x_prev) · bel_prev[x_prev] dx_prev
    for each candidate state x_t:
        bel[x_t] = p(z_t | x_t) · bel_bar[x_t]
      = 1 /   bel[x_t] dx_t
    return   · bel
```

This is **exact** — no approximations. The problem is that the integrals are intractable in continuous state spaces. **Kalman, particle, and histogram filters are three different ways to compute these integrals.**

3 The Kalman Filter

3.1 When to Reach for It

The Kalman filter solves the Bayes filter exactly under three assumptions:

1. The motion model is **linear** in the state and Gaussian:

$$x_t \;=\; A_t\, x_{t-1} + B_t\, u_t + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0,\, R_t).$$

2. The measurement model is **linear** in the state and Gaussian:

$$z_t \;=\; C_t\, x_t + \delta_t, \qquad \delta_t \sim \mathcal{N}(0,\, Q_t).$$

3. The initial belief is Gaussian:

$$\text{bel}(x_0) \;=\; \mathcal{N}(x_0;\, \mu_0,\, \Sigma_0).$$

Symbol	Meaning	Typical size
A_t	State transition matrix	$d \times d$
B_t	Control influence matrix	$d \times m$
C_t	Measurement matrix	$k \times d$
R_t	Process-noise covariance	$d \times d$
Q_t	Measurement-noise covariance	$k \times k$
μ_t	Mean of the belief	$d \times 1$
Σ_t	Covariance of the belief	$d \times d$

Intuition

A Kalman filter assumes the world is a smooth, linear pinball machine where every uncertainty is shaped like a bell curve. The price of those assumptions is enormous speed and a closed-form answer in just five matrix equations.

3.2 The Five Equations

If $\text{bel}(x_{t-1}) = \mathcal{N}(\mu_{t-1}, \Sigma_{t-1})$, then $\text{bel}(x_t) = \mathcal{N}(\mu_t, \Sigma_t)$ where:

Predict:

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \tag{1}$$
$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^\top + R_t \tag{2}$$

Correct:

$$K_t = \bar{\Sigma}_t C_t^\top \left(C_t \bar{\Sigma}_t C_t^\top + Q_t \right)^{-1} \tag{3}$$
$$\mu_t = \bar{\mu}_t + K_t \left(z_t - C_t \bar{\mu}_t \right) \tag{4}$$
$$\Sigma_t = \left(I - K_t C_t \right) \bar{\Sigma}_t \tag{5}$$

The matrix K_t is the **Kalman gain** — how much to trust the measurement vs. the prediction.

Intuition

Predict: push the mean through the dynamics and *inflate* the covariance by the process noise. We trust the dynamics, but they're not perfect, so uncertainty grows.
Correct: compute the residual $z_t - C_t \bar{\mu}_t$ (how surprised are we?). Move the mean toward the measurement by an amount K_t, and *shrink* the covariance because we just gained information. The gain K_t is large when the measurement is precise (small Q_t) and small when the measurement is noisy.

3.3 Why These Are the Right Formulas

The Kalman filter equations follow from one algebraic fact: **the product of two Gaussians is itself a Gaussian**, and you can read off its mean and covariance in closed form. Applying this fact to the Bayes filter's two steps gives exactly the equations above. A full derivation is beyond these notes (see Thrun *et al.*, *Probabilistic Robotics*, Ch. 3).

3.4 Extension: The Extended Kalman Filter (EKF)

Real robots are not linear (a unicycle's heading shows up inside $\sin\theta, \cos\theta$). The **EKF** patches this by **linearizing** f and h around the current mean each step:

$$A_t = \left. \frac{\partial f}{\partial x} \right|_{\mu_{t-1},\, u_t}, \qquad C_t = \left. \frac{\partial h}{\partial x} \right|_{\bar{\mu}_t}. \tag{6}$$

Then it uses the *exact* nonlinear models for the means and the *Jacobians* for the covariances. This is what almost every Kalman-style robot localizer actually uses in practice.

3.5 Strengths and Weaknesses

Strengths

- Tiny memory and CPU footprint: $\mathcal{O}(d^3)$ per step where d is the state dimension.
- Closed-form, no random sampling — fully reproducible.
- Optimal in the mean-squared-error sense under its assumptions.

Weaknesses

- A single Gaussian cannot represent **multimodal** beliefs. If the robot might be at one of two hallways, the Kalman filter pretends it's "somewhere in between" — which is a wall.
- Linearization (EKF) breaks down for strongly nonlinear models or for highly uncertain initial conditions.
- Cannot handle global localization (an initial uniform prior is not Gaussian).

Intuition

The Kalman filter is the right tool when you already roughly know where you are and just want to refine that estimate as the robot moves. It is the wrong tool when you have multiple hypotheses to track at once.

4 The Particle Filter

4.1 The Idea

Drop the Gaussian assumption entirely. Approximate the belief with a **set of samples**:

$$\mathcal{X}_t = \left\{ x_t^{[1]}, x_t^{[2]}, \ldots, x_t^{[N]} \right\}.$$

Each $x_t^{[i]}$ is a complete state hypothesis ("the robot might be here"). The **density of particles** in a region is proportional to the probability the true state is in that region.

Intuition

Instead of writing down a curve $\text{bel}(x_t)$, sprinkle N pins on the map where the curve is high and few where it is low. If you squint, the pin density traces out the curve. As $N \to \infty$, the approximation becomes exact.

4.2 Three Steps per Cycle

Given the previous particle set $\mathcal{X}_{t-1}$, control u_t, and measurement z_t:

4.2.1 Step 1 — Predict (sample from the motion model)

For each $i = 1, \ldots, N$, draw one sample of the next state:

$$x_t^{[i]} \sim p(x_t \mid u_t, x_{t-1}^{[i]}).$$

In code this means: take $x_{t-1}^{[i]}$, apply the deterministic motion equations, then add a **random noise sample** drawn from your motion-noise model.

Intuition

Each particle gets pushed forward by the control, but with its *own* noise sample. Particles that were already close together fan out into a cloud — exactly the "belief spreads" behavior we want.

4.2.2 Step 2 — Weight (score by the measurement model)

For each particle compute an **importance weight**:

$$w^{[i]} \;=\; p(z_t \mid x_t^{[i]}).$$

Then normalize:

$$\tilde{w}^{[i]} \;=\; \frac{w^{[i]}}{\sum_{j=1}^{N} w^{[j]}}.$$

Intuition

Ask each particle: "if you were the robot, how likely is the scan I just saw?" Particles that "see what the robot sees" get high marks; particles that disagree get near-zero scores.

4.2.3 Step 3 — Resample (proportional to weight)

Draw N new particles, **with replacement**, where particle i is selected with probability $\tilde{w}^{[i]}$. The new set replaces $\mathcal{X}_t$, and all weights reset to $1/N$.

Intuition

Multiply the good particles, kill the bad ones. After resampling, the particle density reflects the belief, so every particle counts equally going forward. Without resampling, a few high-weight particles would dominate and the rest would be wasted compute.

4.3 Why It Works — Importance Sampling

The three steps implement an algorithm called **sequential importance resampling**. The clever part is the relationship:

$$\underbrace{\tilde{w}^{[i]}}_{\text{normalized weight}} \;\propto\; \frac{\overbrace{p(z_t \mid x_t^{[i]})}^{\text{sensor model}}\;\overbrace{p(x_t^{[i]} \mid u_t, x_{t-1}^{[i]})}^{\text{motion model (already used in sampling)}}}{\underbrace{p(x_t^{[i]} \mid u_t, x_{t-1}^{[i]})}_{\text{proposal we sampled from}}} \;=\; p(z_t \mid x_t^{[i]}).$$

Because we sampled from the motion model, the motion factor in numerator and denominator cancel — leaving only the measurement model. **That's why each particle's weight is just its sensor likelihood**, and that's why the algorithm has only three steps.

4.4 Practical Refinements

1. **Effective sample size.** Only resample when

$$N_{\text{eff}} \;=\; \frac{1}{\sum_i (\tilde{w}^{[i]})^2}$$

   drops below a threshold (e.g. $N/2$). This preserves diversity.
2. **Low-variance resampling.** Replace N independent draws with a single random offset stepping through the CDF — same expected behavior, lower variance.
3. **Augmented MCL.** Inject a small fraction of uniformly random particles each step to recover from "kidnapping."

4.5 Strengths and Weaknesses

Strengths

- Handles arbitrary belief shapes — multimodal, skewed, anything.
- Works with any motion or measurement model — no linearization required.
- Easy to implement (the whole thing is about 20 lines of substance).

Weaknesses

- Cost is $\mathcal{O}(N)$ per step times the cost of one sensor-model evaluation.
- Variance — different runs give slightly different beliefs.
- **Particle deprivation** if N is too small: the true state has zero particles and the filter can't recover.

Intuition

Think of particle filters as "a small parliament voting on where the robot is." Predict moves every member's opinion forward, weight scores each opinion against the latest evidence, and resample makes the parliament more representative of well-supported opinions.

5 Monte Carlo Localization (MCL)

5.1 What MCL Is

Monte Carlo Localization is the particle filter, applied to robot localization on a known map. That's it — no new equations, just a specific instantiation:

- **State** $x_t = (x, y, \theta)$, the robot pose.
- **Motion model** $p(x_t \mid u_t, x_{t-1})$: the unicycle (or differential-drive) equations from W09, plus a noise model that scales with the commanded motion.
- **Measurement model** $p(z_t \mid x_t)$: for a LiDAR, simulate the scan a robot at pose x_t would see using **ray-casting against the known map** (we built this in W10), then score each beam.

5.2 The Measurement Model in Detail

The standard "beam-based" model for a K-beam LiDAR is:

$$p(z_t \mid x_t) \;=\; \prod_{k=1}^{K} p\big(z_t^{(k)} \,\big|\, \hat{z}^{(k)}(x_t, \text{map})\big),$$

where $z_t^{(k)}$ is the actual range returned by beam k, and $\hat{z}^{(k)}$ is the expected range obtained by **ray-casting** beam k from pose x_t until it hits an obstacle on the map.

For each beam a simple Gaussian likelihood is:

$$p(z_t^{(k)} \mid \hat{z}^{(k)}) \;\propto\; \exp\left[-\frac{(z_t^{(k)} - \hat{z}^{(k)})^2}{2\sigma^2}\right].$$

In practice the beam model adds extra terms for unexpected obstacles, max-range readings, and random noise — but the Gaussian residual is the core.

Intuition

For each particle, *pretend* the robot is there and simulate what the LiDAR would have returned. If the simulated scan matches the actual scan, the particle is plausible.

5.3 Global Localization (the Kidnapped Robot)

Because the particle filter naturally handles multimodal beliefs, MCL can be initialized **uniformly over the free space of the map** — no prior pose required. After a handful of scan-plus-motion cycles, the particle cloud collapses onto the true pose. This is the famous demo from Sebastian Thrun's lab (CMU, late 1990s) that made MCL a textbook method.

A failure mode: if a malicious operator picks up the robot mid-run and drops it somewhere else (the "kidnapped robot problem"), the existing particles all become bad. **Augmented MCL** prevents this by always keeping a small fraction of random samples alive — they catch the robot when it teleports.

5.4 The MCL Algorithm

```
Input:  particles (each is a pose (x, y, theta)),
        control u_t, measurement z_t (K beam ranges),
        map (occupancy grid)

# 1. Predict - sample each particle's next pose
for i in 1..N:
    particles[i] = motion_model_sample(particles[i], u_t)

# 2. Weight - likelihood of z_t under each particle
for i in 1..N:
    z_expected   = raycast(map, particles[i])
    weights[i]   =  _k  exp( -(z_t[k] - z_expected[k])^2 / (2  ^2) )

# 3. Resample only if effective sample size is low
if N_eff(weights) < N/2:
    particles = low_variance_resample(particles, weights)

return particles
```

5.5 Putting It All Together

The table below shows how the four methods relate (already covered in the slides — repeated here for the written record):

Method	Belief shape	Problem
Bayes filter	(the master math)	(the master problem)
Kalman / EKF	One Gaussian	Localization with a smooth, unimodal estimate
Particle filter	N samples	Same Bayes filter, multimodal-friendly representation
MCL	N samples	Particle filter specialized for robot localization with a known map

Intuition

Think of the Bayes filter as a recipe and the others as different kitchens. Kalman: a French kitchen — exact, clean, but only certain ingredients allowed. Particle filter: a hot pot — anything goes, but you need a lot of ingredients (particles). MCL: that same hot pot, used specifically to cook "robot on a map."

6 A Tiny Worked Example

Consider a 1-D robot living on a corridor $0 \le x \le 10$ m. The wall is at $x = 10$. A sensor reports distance-to-wall with Gaussian noise of std-dev $\sigma = 1$ m. We have three particles:

$$x^{[1]} = 2.0, \quad x^{[2]} = 5.0, \quad x^{[3]} = 8.0, \qquad w^{[1]} = w^{[2]} = w^{[3]} = \tfrac{1}{3}.$$

The robot applies control $u_t = +1.0$ m (noise-free for this exercise) and reads $z_t = 3.0$ m.

Step 1 — Predict. Each particle moves +1:

i	$x_{t-1}^{[i]}$	$x_t^{[i]}$
1	2.0	3.0
2	5.0	6.0
3	8.0	9.0

Step 2 — Weight. Expected reading $\hat{z}^{[i]} = 10 - x_t^{[i]}$, error $e^{[i]} = z_t - \hat{z}^{[i]}$, weight $\propto \exp[-\frac{1}{2}(e^{[i]}/\sigma)^2]$:

i	$x_t^{[i]}$	$\hat{z}^{[i]}$	$e^{[i]}$	$w^{[i]}$	normalized
1	3.0	7.0	-4.0	3.4×10^{-4}	0.018
2	6.0	4.0	-1.0	0.607	0.965
3	9.0	1.0	$+2.0$	0.135	0.018

Step 3 — Resample. Three multinomial draws with these probabilities almost certainly produce three copies of particle 2 ($x = 6.0$). In a real filter, motion noise at the next predict step will spread them out into a tight cloud around the true pose.

Intuition

One scan + the right motion model + three particles "found" the robot. With many more particles and a 2-D map, the same arithmetic — repeated 10 times per second — is what powers a real Roomba.

7 Where to Go Next

- **SLAM** (covered in the slides): re-run all of the above with an augmented state $\xi_t = [x_t\,;$ landmarks]. Same math, bigger state.
- **Probabilistic Robotics** by Thrun, Burgard, and Fox is the canonical reference for everything in these notes.
- **ROS** packages: `amcl` (MCL implementation), `gmapping` (FastSLAM), `cartographer` (pose-graph SLAM).

Intuition

Once you understand the Bayes filter you "own" the whole family — every named method is one (representation, problem) pair, and the equations always boil down to "predict, then correct."